RSA

1) Geração das chaves

p e **q** : função *gera_primo*

- ✓ função *secrets.randbelow* – gera número inteiro aleatório no intervalo $[2^{1024}, 2^{1025})$
- ✓ função *aplica_miller_rebin* – realiza teste probabilístico de primalidade

n : n = p.q

e : função *gera_e*

- ✓ inicializa **e** como 65537 e considera $\boldsymbol{\phi}(\mathbf{n})$ = (p-1)(q-1)
- ✓ checa se **e** e $\boldsymbol{\phi}(\mathbf{n})$ são coprimos (mdc = 1)
- ✓ em caso negativo, testa próximo número ímpar até encontrar **e** coprimo de $\boldsymbol{\phi}(\mathbf{n})$

d : função *pow* (nativa do Python)

- ✓ calcula **d** como o inverso de **e** módulo $\boldsymbol{\phi}(\mathbf{n})$, isto é, $\mathbf{d.e} \equiv \mathbf{1} \pmod{\boldsymbol{\phi}(\mathbf{n})}$

Brasília, 5 maio 2026

160023629: Ana Paula Gomes de Matos | 202033427: André Milhomem A. Godoi | 231006239: Geilson dos Santos Sa

RSA

2) Cifragem e decifragem de mensagens

- função *cifra_decifra* (invoca função *pow*)
  - ✓ cifrar: $\mathbf{c} = \mathbf{m}^{\mathbf{e}} \bmod \mathbf{n}$ (chave pública do destinatário)
  - ✓ decifrar: $\mathbf{m} = \mathbf{c}^{\mathbf{d}} \bmod \mathbf{n}$ (chave privada do destinatário)
  - ✓ premissa: para **m** < **n**, a cifragem é invertível pela decifragem (fundamento: Teorema de Euler)

3) Geração e verificação de assinatura

- funções *assina_mensagem*, *empacota_pss* e *desempacota_pss*:
  - ✓ gerar *hash:* **h** = *hashlib.sha3_256*(**m**)
  - ✓ cifrar *hash*: $\mathbf{a} = \mathbf{h}^{\mathbf{d}} \bmod \mathbf{n}$ (chave privada do remetente)
  - ✓ compartilhar mensagem e *hash* cifrado (**m** e **a**)
  - ✓ decifrar *hash*: **h'** = $\mathbf{a}^{\mathbf{e}} \bmod \mathbf{n}$ (chave pública do remetente)
  - ✓ verificar *hash:* **h'** = **h** ?

Brasília, 5 maio 2026

160023629: Ana Paula Gomes de Matos | 202033427: André Milhomem A. Godoi | 231006239: Geilson dos Santos Sa

4) Esquemas OAEP e PSS

- esquemas complementares ao RSA, previstos na RFC 8017
- OAEP → mensagens; PSS → assinaturas
- objetivo: adicionar elemento de aleatoriedade ao RSA (*seed* no OAEP, *salt* no PSS)
- esquemas aplicados antes da cifragem, que passa a incidir sobre *encoded message* (**em**), não diretamente sobre a mensagem original (**m**) ou o *hash* (**h**)
- funções *empacota_oaep, desempacota_oaep, empacota_pss, desempacota_pss*
- elementos:
  - ✓ função *secrets.token_bytes*: gera componentes aleatórios (*seed* e *salt*)
  - ✓ função *aplica_mgf1*: função geradora de máscaras (para operações *XOR*)

RSA

4) Esquemas OAEP e PSS

```
                    +----------+------+--+-------+
           DB =     |  lHash   |  PS  |01|   M   |
                    +----------+------+--+-------+
                                   |
      +----------+                 |
      |   seed   |                 |
      +----------+                 |
            |                      |
            |-------> MGF ---> xor
            |                      |
   +--+     V                      |
   |00|    xor <----- MGF <-----|
   +--+     |                      |
     |      |                      |
     V      V                      V
   +--+----------+----------------------------+
EM =  |00|maskedSeed|          maskedDB          |
   +--+----------+----------------------------+
```

OAEP (mensagens)

```
                          +-----------+
                          |     M     |
                          +-----------+
                                |
                                V
                              Hash
                                |
                                V
                   +--------+----------+----------+
          M' =     |Padding1|  mHash   |   salt   |
                   +--------+----------+----------+
                                |
          +--------+----------+  V
   DB =   |Padding2|   salt   |  Hash
          +--------+----------+   |
                    |            |
                    V            |
                   xor <--- MGF <---|
                    |            |
                    |            |
                    V            V
          +-------------------+----------+--+
   EM =   |    maskedDB       |     H    |bc|
          +-------------------+----------+--+
```

PSS (assinaturas)

Brasília, 5 maio 2026

160023629: Ana Paula Gomes de Matos | 202033427: André Milhomem A. Godoi | 231006239: Geilson dos Santos Sa

5) Resultados e conclusões

Tabela II
TEMPO MÉDIO DE CPU

Operações	Iterações	Tempo médio de CPU
Geração de primo	100	3584,84 ms
Codificação OAEP	10000	0,00234 ms
Decodificação OAEP	10000	0,04531 ms
Cifragem RSA (mensagem)	10000	0,16406 ms
Cifragem RSA (assinatura)	100	33,125 ms
Decifragem RSA (mensagem)	100	21,71875 ms
Decifragem RSA (assinatura)	10000	0,19375 ms
Codificação PSS	10000	0,0375 ms
Decodificação PSS	10000	0,03906 ms

- A geração de chaves e a cifragem/decifragem (para **d** ou **e** grandes) são computacionalmente custosas.
- No RSA, há limitação significativa do tamanho das mensagens (porque **m** < **n**).
- Os esquemas OAEP e PSS adicionam uma camada de segurança ao RSA, mas consomem espaço útil das mensagens.

repositório: github.com/andmilhomem/rsa

Brasília, 5 maio 2026

160023629: Ana Paula Gomes de Matos | 202033427: André Milhomem A. Godoi | 231006239: Geilson dos Santos Sa